Experiment 10

Student Name: Harshit Kumawat
UID: 24BAI70025
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1

Semester: 4
Date of Performance: 24/4/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

- To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

Problem Statement

- In enterprise databases, primary keys must be unique for every record. Manual assignment of keys can lead to errors.

Design a trigger that:

- Automatically generates or enforces a primary key value during record insertion.
- Implements the logic similar to a stored procedure.
- Demonstrates automated primary key functionality for a table.

(Industry relevance: Amazon, Flipkart, Oracle)

Practical/Experiment Steps

- Table Setup: Created the target table (orders) without a default primary key generator to simulate the manual key assignment problem.
- Sequence Creation: Defined an Oracle sequence (order_seq) to serve as the reliable counter for unique numeric IDs.
- Trigger Design: Designed a BEFORE INSERT trigger (trg_order_pk) on the orders table to intercept each insertion event.
- Auto-Key Assignment: Implemented logic inside the trigger to fetch the next value from the sequence and assign it to :NEW.order_id, overriding any user-supplied or NULL value.
- Insertion Testing: Executed multiple INSERT statements without specifying an order_id, relying entirely on the trigger for key generation.
- Output Verification: Queried the orders table after all insertions to confirm unique, sequential primary key values were assigned automatically.

Procedure

- Connected to Oracle/PostgreSQL environment and enabled DBMS output console.
- Created the orders table with order_id as the primary key column (no DEFAULT or auto-increment).
- Created a sequence order_seq starting from 1 with an increment of 1.
- Wrote and compiled the BEFORE INSERT trigger trg_order_pk to populate :NEW.order_id using order_seq.NEXTVAL.
- Inserted multiple rows into orders without specifying order_id.
- Verified that all rows received unique, auto-generated primary key values by running a SELECT query.

Input/Output Analysis

SQL Input Queries

```
CREATE TABLE orders (
    order_id   NUMBER PRIMARY KEY,
    item_name  VARCHAR2(100),
    quantity   NUMBER,
    order_date DATE
);
```

Output

```
SQL> CREATE TABLE orders (
        order_id   NUMBER PRIMARY KEY,
        item_name  VARCHAR2(100),
        quantity   NUMBER,...
Show more...



Table ORDERS created.

Elapsed: 00:00:00.020
```

SQL Input Queries

```
CREATE SEQUENCE order_seq
    START WITH 1
    INCREMENT BY 1
    NOCACHE
    NOCYCLE;
```

Output

```
SQL> CREATE SEQUENCE order_seq
        START WITH 1
        INCREMENT BY 1
        NOCACHE...
Show more...



Sequence ORDER_SEQ created.

Elapsed: 00:00:00.007
```

SQL Input Queries

```
CREATE OR REPLACE TRIGGER trg_order_pk
BEFORE INSERT ON orders
```

```
FOR EACH ROW
BEGIN
    IF :NEW.order_id IS NULL THEN
        :NEW.order_id := order_seq.NEXTVAL;
    END IF;
END;
/
```

Output

```
SQL> CREATE OR REPLACE TRIGGER trg_order_pk
     BEFORE INSERT ON orders
     FOR EACH ROW
     BEGIN...
Show more...



Trigger TRG_ORDER_PK compiled

Elapsed: 00:00:00.026
```

SQL Input Queries

```
INSERT INTO orders (item_name, quantity, order_date)
    VALUES ('Laptop', 2, SYSDATE);

INSERT INTO orders (item_name, quantity, order_date)
    VALUES ('Mobile', 5, SYSDATE);

INSERT INTO orders (item_name, quantity, order_date)
    VALUES ('Tablet', 3, SYSDATE);

COMMIT;
```

Output

```
SQL> INSERT INTO orders (item_name, quantity, order_date)
     VALUES ('Laptop', 2, SYSDATE)



1 row inserted.

Elapsed: 00:00:00.024
```

```
SQL> INSERT INTO orders (item_name, quantity, order_date)
     VALUES ('Mobile', 5, SYSDATE)



1 row inserted.

Elapsed: 00:00:00.002
```

```
SQL> INSERT INTO orders (item_name, quantity, order_date)
     VALUES ('Tablet', 3, SYSDATE)



1 row inserted.

Elapsed: 00:00:00.002
```

SQL Input Queries

```
SELECT order_id, item_name, quantity, order_date FROM orders;
```

Output

```
SQL> SELECT order_id, item_name, quantity, order_date FROM orders

ORDER_ID ITEM_NAME QUANTITY ORDER_DATE
-------- --------- -------- -------------------------
1        Laptop    2        04/24/2026, 11:41:42 AM
2        Mobile    5        04/24/2026, 11:41:42 AM
3        Tablet    3        04/24/2026, 11:41:42 AM

Elapsed: 00:00:00.122
3 rows selected.
```

Learning Outcomes

- Trigger Purpose and Working: Understood the purpose and event-driven nature of database triggers, including BEFORE INSERT, AFTER INSERT, and other trigger types.
- Automated Primary Key Functionality: Learned to implement automated primary key generation using triggers combined with sequences, eliminating manual key management.
- Data Integrity Enforcement: Gained expertise in ensuring data integrity automatically without relying on application-level logic or manual key assignment.
- Encapsulation and Security: Applied trigger-based automation relevant to real-world enterprise applications such as Amazon order management, Flipkart inventory systems, and Oracle ERP solutions.